DA5402 End To End Project

Govind Harsha Vardhan DA25S018

CLIP Product Retrieval, Continual Learning MLOps Pipeline Report

1. Introduction and Motivation

E-commerce platforms maintain product catalogues that evolve on a weekly/periodic basis — new products arrive, prices change, and category distributions shift. A retrieval system trained on a snapshot of this catalogue will gradually degrade as the product distribution drifts away from its training data. Full retraining of large vision-language models on every data update is computationally expensive and often unnecessary when changes are incremental.

This report describes an end-to-end MLOps pipeline that addresses this challenge. The system ingests weekly E Commerce product CSVs, detects whether the embedding distribution has shifted enough to justify retraining, and — if so — adapts a lightweight projection head on top of a frozen CLIP backbone using a replay buffer that prevents catastrophic forgetting. All stages are orchestrated by Apache Airflow, tracked in MLflow, versioned with DVC, and served via a FAISS-backed FastAPI service.

Core design principle. Rather than retraining the full CLIP model (ViT-B/32, $\sim$150M parameters) weekly, only a 2×512 linear projection head is trained. Retraining is gated by a weighted drift score combining embedding shift, Recall degradation, and Consistency degradation (Defined in Section 4.5). This reduces compute by roughly two orders of magnitude while preserving retrieval quality.

2. System Architecture

2.1 Three-Plane Architecture

The system is organised into three functional planes with explicit interfaces between them, enabling loose coupling and independent operation.

Training Plane. Receives raw product CSVs and is responsible for all data processing, model adaptation, and artifact production. Implemented as an Apache Airflow DAG that runs on a 5-minute polling schedule, with each run being fully idempotent.

Serving Plane. Consumes trained artifacts from the MLflow Model Registry and responds to search queries in real time. Consists of two containers: `mlflow-serve` (pyfunc model server on port 5001) and `search-api` (FastAPI gateway on port 8000).

Data/State Plane. Manages all persistent state: raw CSVs, augmented datasets, image caches, FAISS indices, and embedding files. DVC tracks data artifacts; Git tracks code and configuration; MLflow tracks model artifacts and metrics. Together, these three systems enable complete point-in-time reproducibility.

2.2 Airflow DAG Task Graph

The pipeline is defined as a single Airflow DAG (`clip_product_retrieval_pipeline`) with eleven tasks. The critical branching point is `branch_baseline`: on the first run (Day-

0), it routes to `initial_train`; on subsequent runs (Week N), it routes to `drift_check`, which either triggers `retrain` or `skip_retrain`. Both branches converge at `update_faiss`, after which provenance is logged, the model is registered, and the serving layer is reloaded.

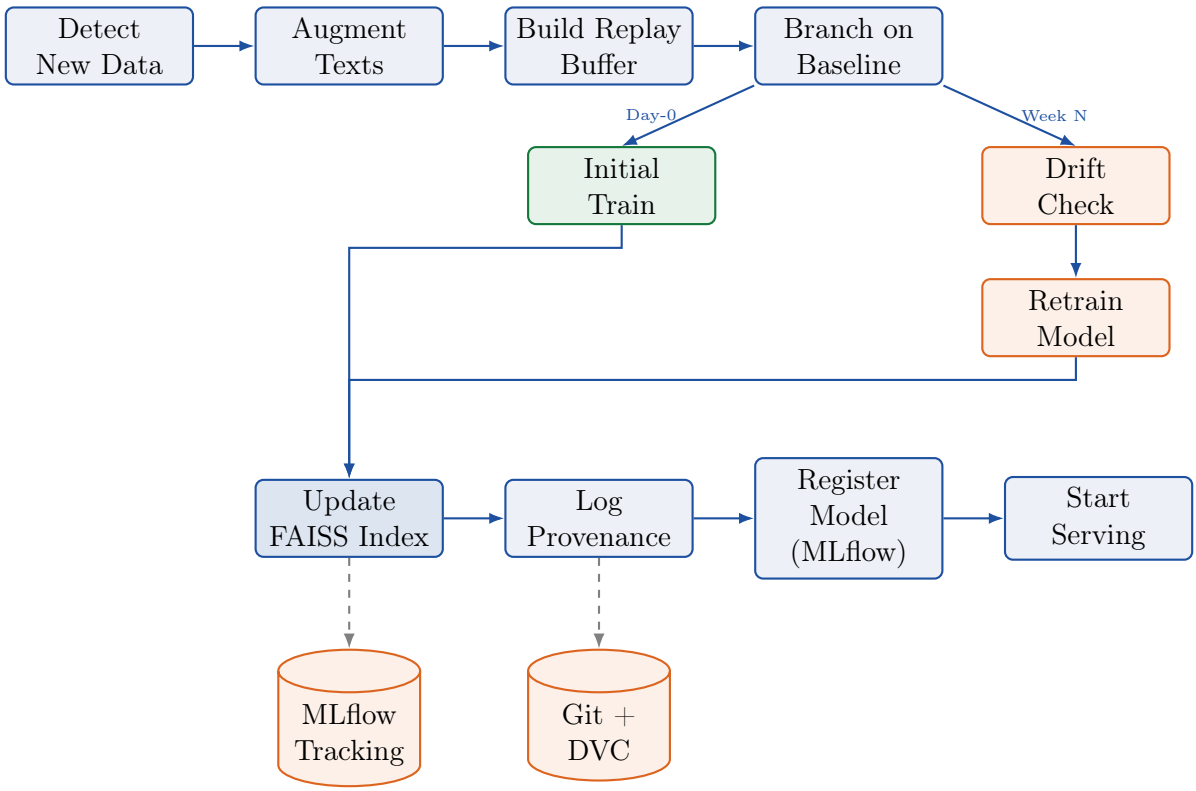


Figure 1: Airflow DAG task dependency graph. The pipeline branches at `branch_baseline`: Day-0 performs initial training; subsequent runs check for drift and retrain only if the score exceeds the threshold. Both branches converge at `update_faiss`, after which artifacts are registered and the serving layer is reloaded atomically.

2.3 Deployment Topology

All services run as Docker containers coordinated by Docker Compose. The training and serving layers are fully decoupled — they communicate exclusively through the MLflow Model Registry (training writes; serving reads) and a single `POST /reload` REST call.

Table 1: Docker Compose services and their roles.

Service	**Port**	**Role**
`postgres`	5432	Shared metadata DB for Airflow and MLflow
`mlflow`	5000	Tracking server; artifact store on Docker volume
`airflow-webserver`	8080	Airflow UI, REST API, and health endpoints
`airflow-scheduler`	—	DAG scheduling and task dispatch
`mlflow-serve`	5001	Serves `clip_product_retrieval@Production` as pyfunc
`search-api`	8000	FastAPI gateway: search, health, reload, pipeline trigger
`prometheus`	9090	Metrics scraping from all services (15 s interval)
`grafana`	3001	Near-real-time dashboard for latency and error rate
`alertmanager`	9093	Alert routing for downtime and degradation events
`postgres-exporter`	9187	Exposes Postgres metrics to Prometheus

3. Data Engineering Pipeline

3.1 Input Schema and Ingestion

The pipeline detects new weekly CSVs by scanning `data/incoming/` for files matching `week`*N*`.csv` that have not yet produced a processed replay CSV. This file-presence check is idempotent — running the DAG multiple times for the same week is safe.

Each weekly CSV must contain three required columns: `h_index` (unique product identifier), `title`, and `image_links`. Five additional columns (`category_2`, `category_3`, `selling_price`, `highlights`, `description`) are optional and enable richer text augmentation when present.

3.2 Text Augmentation

The augmentation module (`src/augment_texts.py`) converts each product record into up to seven text variants, all sharing the same `original_index`. This gives the contrastive training loss multiple text supervision signals per image, improving cross-modal alignment on the domain-specific vocabulary of e-commerce product titles.

Table 2: The seven text augmentation strategies applied to each product.

Strategy	**Description**
`title_clean`	Remove noise characters (NBSP, brackets, pipes); normalise whitespace. Used as the canonical evaluation text.
`category_prefix`	Prepend `category_2 > category_3:` to the cleaned title.
`price_context`	Natural-language price phrase, e.g. *"mid-range product under Rs. 45000"*.
`highlights_fusion`	Title keywords concatenated with parsed attribute values from the highlights field.
`description_snippet`	First 25 words of the product description (suppressed if >70% overlap with title).
`keyword_drop`	Title with 30% of non-stopwords randomly removed (augmentation for robustness).
`attribute_reorder`	Colour and size attributes shuffled within the title to encourage order-invariant representations.

After augmentation, records are deduplicated on the `(original_index, augmented_text)` pair using a method-priority ordering that retains the most informative variant in case of collision.

3.3 Replay Buffer for Continual Learning

The replay buffer (`src/replay_buffer.py`) is the mechanism by which the system prevents catastrophic forgetting. When training on week N, the buffer mixes current-week data with a strategically sampled subset from week $N-1$.

Given that the new dataset has n_{new} images and the replay ratio $r = 0.20$:

$$n_{\text{replay}} = \max(2,\ \lfloor n_{\text{new}} \cdot r \rfloor) \tag{1}$$

$$n_{\text{random}} = \lfloor n_{\text{replay}}/2 \rfloor \quad \text{(randomly sampled from previous week)} \tag{2}$$

$$n_{\text{hard}} = n_{\text{replay}} - n_{\text{random}} \quad \text{(hard negatives, see below)} \tag{3}$$

Hard negative mining. Old products are ranked by cosine similarity to the baseline text centroid; the n_{hard} products with the *lowest* similarity are selected. These are the products most dissimilar from the current training distribution and therefore most at risk of being forgotten during adaptation.

To prevent `original_index` collisions between new data and replayed data, non-cumulative index offsets are applied: random replay samples receive an offset of $10^6 \times \text{week_number}$, and hard negatives receive $2 \times 10^6 \times \text{week_number}$.

4. Model Design

4.1 CLIP Backbone and Operating Modes

The system uses OpenCLIP ViT-B/32 with OpenAI pre-trained weights. The backbone produces 512-dimensional, L2-normalised embeddings for both images and text. Three operating modes are supported:

- **Zero-shot.** Plain CLIP with no adaptation. Used as the baseline evaluation point against which drift is measured.
- **Linear probe (default).** Frozen backbone with trainable 512×512 projection heads for image and text. Trains in minutes on a CPU.
- **Fine-tune.** Last N transformer blocks unfrozen in addition to the projection heads. Provides higher adaptation capacity at greater compute cost.

4.2 Linear Probe Architecture

The projection head applies a linear transformation to the frozen CLIP embeddings, followed by L2-normalisation:

$$\hat{v}^{\text{img}} = \frac{W_{\text{img}}\, \phi_{\text{img}}(x)}{\|W_{\text{img}}\, \phi_{\text{img}}(x)\|_2}, \qquad \hat{v}^{\text{txt}} = \frac{W_{\text{txt}}\, \phi_{\text{txt}}(t)}{\|W_{\text{txt}}\, \phi_{\text{txt}}(t)\|_2} \tag{4}$$

where ϕ_{img}, ϕ_{txt} are the frozen CLIP encoders, and $W_{\text{img}}, W_{\text{txt}} \in \mathbb{R}^{512\times512}$ are initialised to the identity matrix. Identity initialisation ensures the model begins as a valid zero-shot retrieval system and only diverges as training provides evidence that domain adaptation is beneficial.

4.3 InfoNCE Loss

Training uses the symmetric InfoNCE (contrastive) loss over a mini-batch of N image-text pairs:

$$\mathcal{L} = \frac{1}{2}\Big[\mathcal{L}_{\text{img}\to\text{txt}} + \mathcal{L}_{\text{txt}\to\text{img}}\Big] \tag{5}$$

$$\mathcal{L}_{\text{img}\to\text{txt}} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp(\tau\, \hat{v}_i^{\text{img}} \cdot \hat{v}_i^{\text{txt}})}{\sum_{j=1}^{N} \exp(\tau\, \hat{v}_i^{\text{img}} \cdot \hat{v}_j^{\text{txt}})} \tag{6}$$

where $\tau = \exp(\texttt{logit_scale})$ is the learnable temperature shared from the CLIP backbone, clamped at $\tau \leq 100$ to prevent numerical instability.

4.4 Training Configuration

Table 3: Default training hyperparameters (configurable via `params.yaml`).

Hyperparameter	**Value**
Optimiser	AdamW
Learning rate	1×10^{-3}
Weight decay	1×10^{-4}
Batch size	32
Epochs (Airflow DAG)	2
Warmup epochs	1
Gradient clip	1.0
LR schedule	Cosine decay with linear warmup
Validation split	15% (split by `original_index` to prevent leakage)

4.5 Evaluation Metrics

Two complementary retrieval metrics are computed on the validation set after each epoch.

Recall@K measures whether the correct text is retrieved in the top-K results when querying by image embedding:

$$\text{Recall@}K = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\Big[i \in \text{top-}K\Big(\hat{v}_i^{\text{img}} \cdot \hat{V}^{\text{txt}}\Big)\Big] \tag{7}$$

Consistency@K measures cross-modal alignment beyond the correct pairs, via the Jaccard overlap between the image-nearest-neighbour set and text-nearest-neighbour set for each product:

$$\text{Consistency@}K = \frac{1}{N}\sum_{i=1}^{N} \frac{\left|\mathcal{N}_K^{\text{img}}(i) \cap \mathcal{N}_K^{\text{txt}}(i)\right|}{K} \tag{8}$$

Both metrics are computed for $K \in \{1, 5, 10\}$.

5. Drift Detection and Continual Learning Logic

5.1 Motivation

Weekly data changes are not always large enough to justify the cost of retraining. The drift detection module (`src/drift_detection.py`) computes a scalar drift score in $[0, 1]$ by comparing the new week's embeddings and retrieval metrics against a stored baseline. If the score falls below 0.30, retraining is skipped and the existing model continues serving.

5.2 Drift Score Formula

The score combines three signals with fixed weights reflecting the relative importance of each indicator:

$$\text{drift_score} = 0.20 \cdot s_{\text{emb}} + 0.40 \cdot s_{\text{recall}} + 0.40 \cdot s_{\text{consist}} \tag{9}$$

The embedding signal s_{emb} captures distributional shift in the representation space. It is the mean of four sub-signals, computed for both text and image embeddings:

$$s_{\text{centroid}} = \text{clip}(1 - \cos(\bar{e}_{\text{new}},\ \bar{e}_{\text{baseline}}),\ 0, 1) \tag{10}$$

$$s_{\text{pairwise}} = \text{clip}\left(\frac{|\overline{\text{sim}}_{\text{new}} - \overline{\text{sim}}_{\text{baseline}}|}{0.20},\ 0, 1\right) \tag{11}$$

The recall and consistency signals measure relative metric degradation against the baseline:

$$\text{relative_drop}(b, n) = \text{clip}\left(\frac{b - n}{b},\ 0, 1\right) \tag{12}$$

$$s_{\text{recall}} = \tfrac{1}{2}\left[\text{relative_drop}(\text{R@1}_{\text{bl}},\ \text{R@1}_{\text{new}}) + \text{relative_drop}(\text{R@5}_{\text{bl}},\ \text{R@5}_{\text{new}})\right] \tag{13}$$

The decision rule is simply:

$$\text{drift_detected} = \begin{cases} \texttt{True} & \text{if drift_score} \geq 0.30 \\ \texttt{False} & \text{otherwise} \end{cases} \tag{14}$$

The weights assign 80% of the score to performance-based signals and only 20% to embedding-distance. This reflects the insight that small shifts in embedding space may not translate to meaningful retrieval degradation, avoiding unnecessary retraining due to noise in embedding statistics.

6. Vector Search and Serving

6.1 FAISS Index

After each pipeline run, two FAISS indices are rebuilt over the full cumulative corpus using the latest trained model's embeddings: `index_img.bin` for image queries and `index_txt.bin` for text queries. Both use `IndexFlatIP` (exact inner-product search on L2-normalised vectors, equivalent to cosine similarity). A parallel `meta.json` list provides product metadata for result rendering. The invariant `index.ntotal == len(meta)` is enforced at load time; any violation raises `RuntimeError` before a corrupted index can be served.

6.2 Hot-Swap Protocol

The search API updates its FAISS index without restarting via `POST /reload`. The swap uses a threading lock for atomicity:

```
def reload_index() -> int:
    new_index, new_meta = _load()     # load from disk (outside
        the lock)
    with _lock:                       # atomic pointer swap (
        inside the lock)
        _index = new_index
        _meta  = new_meta
    return new_index.ntotal
```

In-flight requests using the old index complete normally; new requests after the lock release see the updated index. This is safe because `IndexFlatIP` is read-only after construction. The Airflow DAG's final task calls `POST /reload` automatically after every successful training run, ensuring zero-downtime updates.

6.3 Search API Endpoints

Table 4: FastAPI search endpoints and supported query modalities.

Endpoint	Input	Embedding
`POST /search/text`	`{"query": "...", "top_k": 10}`	CLIP text encoder
`POST /search/image`	`{"image_b64": "...", "top_k": 10}`	CLIP image encoder
`POST /search/multimodal`	query string + `image_b64`	Averaged and re-normalised
`GET /health`	—	Liveness check
`GET /ready`	—	Readiness check
`POST /reload`	—	FAISS hot-swap

7. MLflow Integration

7.1 Experiment and Run Tracking

All training runs are logged to the `clip_product_retrieval` experiment. Each run records training hyperparameters, per-epoch metrics (Recall@K, Consistency@K, training loss), and metadata tags identifying the mode, stage, and week label. Beyond basic autolog, the pipeline explicitly tracks the following additional artifacts:

- **Drift reference:** `drift_reference.json` stores the baseline centroid coordinates and final metric values. This file is the sole input to every future drift check, making the drift decision fully reproducible.
- **Retrieval failure analysis:** `retrieval_failures.csv` records the top-20 image-text pairs with the lowest cosine similarity on the validation set, enabling targeted debugging of hard cases.
- **Provenance records:** `weekN_provenance.json` stores the Git commit hash, DVC lock file state, and all input/output file paths, creating a complete audit trail for every run.

7.2 Artifact Layout

The following artifacts are stored per run in the MLflow artifact store:

```
artifacts/
|-- model/                          # mlflow.pytorch.log_model
```

```
|-- checkpoints/
|   |-- probe_ep2.pt
|   |-- probe_ep4.pt
|   `-- probe_best.pt
|-- drift/
|   |-- drift_reference.json
|   |-- baseline_text_centroid.npy
|   `-- baseline_image_centroid.npy
|-- confusion/
|   `-- retrieval_failures.csv
|-- provenance/
|   `-- weekN_provenance.json
|-- faiss/
|   |-- index_img.bin
|   |-- index_txt.bin
|   `-- meta.json
`-- embeddings/
    `-- corpus/
        |-- corpus_image_embeddings.npy
        `-- corpus_text_embeddings.npy
```

7.3 Model Registry and Serving Handoff

After each successful training run, the model is registered in the MLflow Model Registry as `clip_product_retrieval` and promoted to the **Production** stage. The registered artifact is a `CLIPRetrieverModel` pyfunc that bundles the frozen CLIP backbone, the trained `LinearProbeHead` weights, the FAISS index, and the product metadata. The `mlflow-serve` container serves this model at `models:/clip_product_retrieval@Production` — a stable URI that always points to the latest promoted checkpoint, requiring no configuration change on promotion.

8. Reproducibility

8.1 Three-Layer Reproducibility

Full reproducibility is achieved by composing three versioning systems, each responsible for a distinct category of artifact:

1. **Code (Git).** Every pipeline run tags the repository at the current commit hash as `week`*N*`/run-`*abc12345*. The commit hash is also stored as an MLflow run tag (`git_commit`), creating a bidirectional link between code and training run.
2. **Data (DVC).** The image cache (`data/image_cache/`) is tracked by DVC with a local remote. `dvc.lock` records the MD5 hash of the entire data/ directory snapshot after each run, and this pointer file is committed to Git alongside the week's tag. dvc.lock is only written by dvc repro, which is used for offline verification — the Airflow DAG does not call dvc repro and does not rely on `dvc.lock` for restore.
3. **Model (MLflow).** The MLflow run ID is stored in `params.yaml`. Any past checkpoint can be downloaded via `mlflow.artifacts.download_artifacts(run_id=...)`, enabling rollback to any previously registered model version.

Figure 2: Prometheus UI

8.2 DVC Pipeline Definition

In addition to the Airflow orchestration, a `dvc.yaml` pipeline defines the same three processing stages for offline reproducibility and CI:

Stage	Dependencies	Output
`augment`	`augment_texts.py`, `weekN.csv`	`weekN_augmented.csv`
`replay`	`replay_buffer.py`, `weekN_augmented.csv`	`weekN_replay.csv`
`train`	`clip_experiments.py`, `weekN_replay.csv`	`last_run_id.txt`

Table 5: DVC pipeline stages and their inputs/outputs.

State restoration for any past week is encapsulated in a single script:

```
# Restore week2's exact state (code + data + model reference)
./reproduce.sh week2/run-abc12345
# Internally: git checkout week2/run-abc12345 && dvc pull --
    force
```

9. Monitoring and Alerting

9.1 Prometheus Instrumentation

The search API is instrumented with `prometheus_fastapi_instrumentator`, which exposes request count, latency histograms, and error counts at `GET /metrics`. Prometheus scrapes all services every 15 seconds. Four alert rules are configured:

Table 6: Prometheus alert rules.

Alert	Condition	Severity
`SearchApiDown`	Service unreachable for >1 min	Critical
`SearchApiHighLatency`	p95 latency > 2.0 s (5-min window)	Warning
`SearchApiHighErrorRate`	Error rate > 5% (5-min window)	Warning
`PostgresDown`	postgres_exporter unreachable for >1 min	Critical

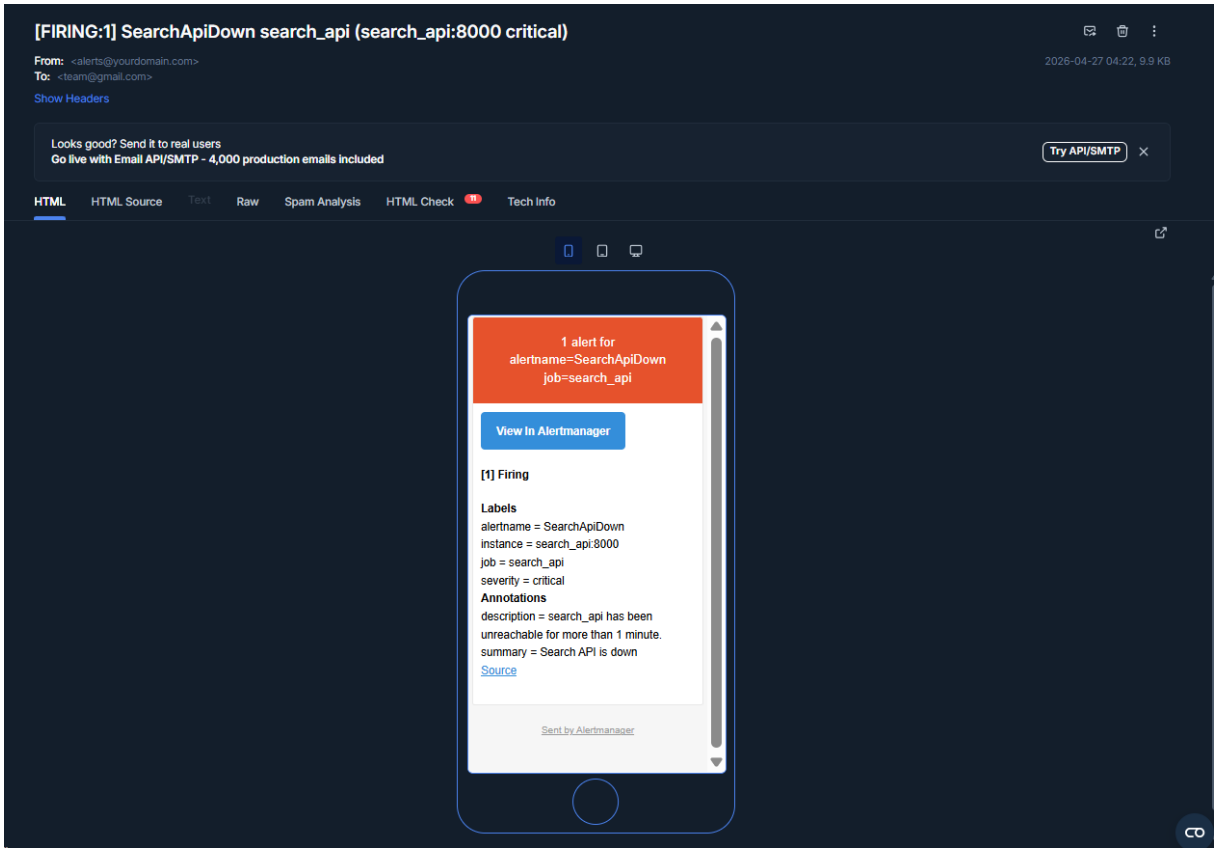


Figure 3: SearchApi Down alert to MailTrap

9.2 Grafana Dashboard

Grafana (port 3001) is provisioned automatically from `monitoring/grafana/dashboards/dashboard.j` The dashboard displays request throughput, p50/p95/p99 latency, error rate, and Postgres connection pool utilisation. Both the data source and dashboard are provisioned via the Docker Compose volume mount, requiring no manual configuration after startup.

10. Pipeline Visualisation and Management

The Airflow webserver (port 8080) provides a full pipeline management console. Figure **??** shows the DAG task graph as rendered by the Airflow UI. Each task's state (queued, running, succeeded, failed, skipped) is visible in real time, and task logs are accessible directly from the graph view. The branching at `branch_baseline` and the converging paths are clearly visible.

The DAG can be triggered in three ways: automatically on the 5-minute polling schedule, manually from the Airflow UI, or programmatically via the FastAPI `POST /pipeline/trigger` endpoint (which wraps the Airflow REST API). This last option allows the search-facing frontend to initiate a retraining run without direct access to the Airflow instance.

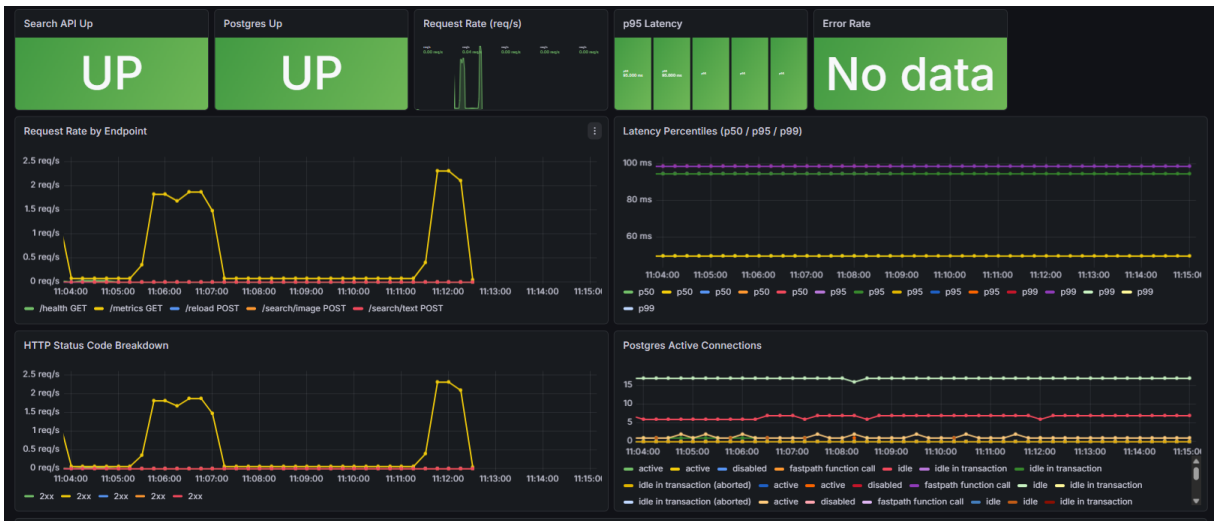


Figure 4: Graphana dashboard

Figure 5: Airflow UI showing the `clip_product_retrieval_pipeline` DAG task graph. Task states update in real time during execution. Green indicates success; light blue indicates a skipped branch (e.g. `skip_retrain` when drift is not detected).

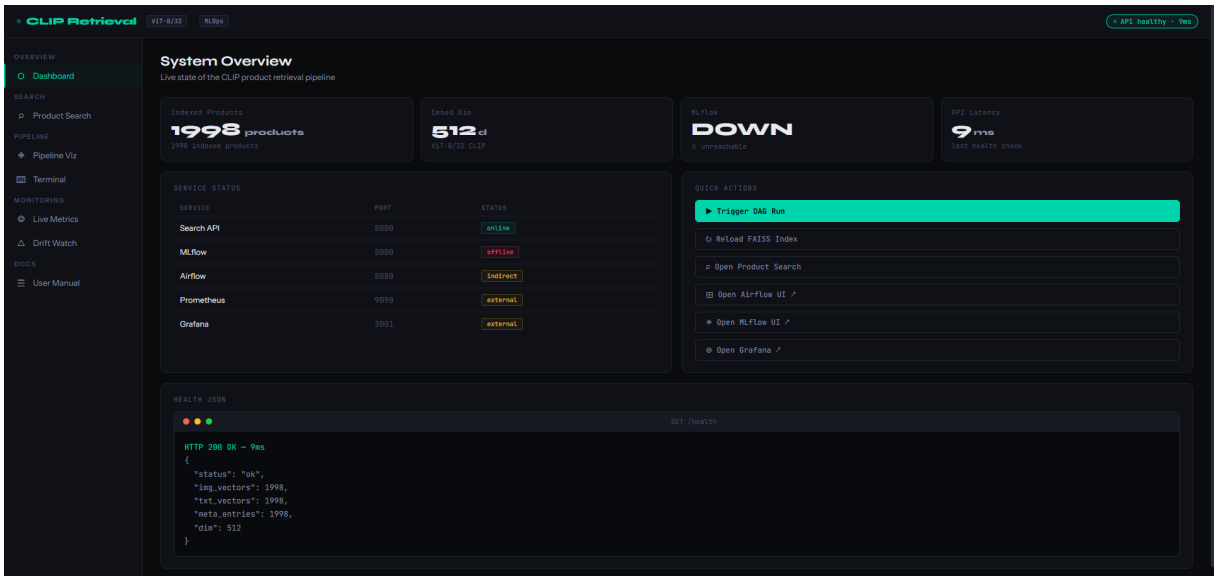
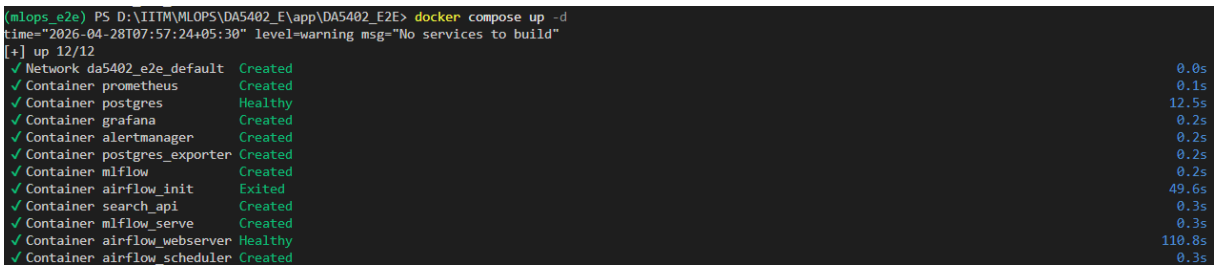


Figure 6: User Interface for Search Engine



```
(mlops_e2e) PS D:\IITM\MLOPS\DA5402_E\app\DA5402_E2E> docker compose up -d
time="2026-04-28T07:57:24+05:30" level=warning msg="No services to build"
[+] up 12/12
 ✓ Network da5402_e2e_default  Created                                    0.0s
 ✓ Container prometheus        Created                                    0.1s
 ✓ Container postgres          Healthy                                   12.5s
 ✓ Container grafana           Created                                    0.2s
 ✓ Container alertmanager      Created                                    0.2s
 ✓ Container postgres_exporter Created                                    0.2s
 ✓ Container mlflow            Created                                    0.2s
 ✓ Container airflow_init      Exited                                    49.6s
 ✓ Container search_api        Created                                    0.3s
 ✓ Container mlflow_serve      Created                                    0.3s
 ✓ Container airflow_webserver Healthy                                  110.8s
 ✓ Container airflow_scheduler Created                                    0.3s
```

Figure 7: Containerized MLOps stack initialized using Docker Compose — all services running and healthy.

11. Software Engineering

11.1 Module Structure and Design Principles

The codebase follows a functional paradigm with strict separation of concerns. Each source module has a single responsibility, and the training and serving layers have no shared imports — they communicate only through the MLflow Model Registry and the `POST /reload` REST call.

Table 7: Source module responsibilities.

Module	Responsibility
`src/augment_texts.py`	Text augmentation; 7 rule-based strategies
`src/clip_experiments.py`	CLIP training modes, evaluation, MLflow logging
`src/drift_detection.py`	Weighted drift score computation
`src/replay_buffer.py`	Continual learning replay buffer construction
`src/faiss_loader.py`	Thread-safe FAISS singleton with hot-swap
`src/register_clip_model.py`	MLflow Model Registry registration
`src/clip_mlflow_wrapper.py`	pyfunc wrapper for MLflow serving
`src/search_api.py`	FastAPI application entry point
`dags/clip_pipeline_dag.py`	Airflow DAG definition (11 tasks)

Comprehensive exception handling is implemented throughout the Airflow tasks: each task catches its specific failure modes, logs structured messages, and re-raises to Airflow so the retry policy (1 retry, 30-second delay) can be applied. The FAISS loader enforces its integrity invariant at load time and raises `RuntimeError` before any corrupted index can be served.

11.2 Testing

Unit tests are provided in `tests/test_search_api.py` and `tests/test_augment_texts.py`, covering the seven augmentation strategies, the deduplication logic, and edge cases such as missing optional columns and malformed input rows.

12. Known Issues and Resolutions

Table 8: Deployment issues encountered during development and their resolutions.

Issue	**Root Cause**	**Resolution**
MLflow `403 Invalid Host header`	MLflow 3.5+ enforces strict host validation; the inter-container hostname `mlflow:5000` is rejected.	Add `MLFLOW_SERVER_ALLOWED_HOSTS: "*"` to the mlflow service environment in `docker-compose.yaml`.
Git `dubious ownership` in `dvc_pull`	`/opt/project` is owned by the host UID; the Airflow container runs as the `airflow` user.	Add `safe.directory /opt/project` in `Dockerfile.airflow` and at runtime inside the `dvc_pull()` task.
`dvc add data/image_cache` blocked	The target path was listed in `.dvcignore`.	Remove `data/image_cache/` from `.dvcignore`.
`dvc pull` fails	Incoming CSVs are DVC pipeline *dependencies*, not *outputs*; DVC cannot pull deps.	Use `dvc pull -force` (pulls all tracked outputs) in the `dvc_pull()` task.
DAG edits not reflected in Airflow UI	Airflow re-serializes DAGs on a polling interval; changes between cycles are invisible in the UI.	Force immediate re-serialization without waiting for the next poll: `docker exec airflow_scheduler airflow dags reserialize`.
`search-api` crashloops on first start	FAISS index files do not exist before the first pipeline run; the eager lifespan loader raises `FileNotFoundError` and uvicorn exits immediately.	Start the serving profile only after the first DAG run completes successfully: `docker compose --profile serving up -d`.
Stale FAISS index after pipeline run	`reload_faiss_index` polls `/health` (liveness), which returns 200 as soon as uvicorn is up — before the index is in memory.	Poll `/ready` instead; it returns 503 until `index.ntotal > 0`, giving a true readiness signal before accepting traffic.

13. Technology Stack Summary

Component	Technology	MLOps Role
Model backbone	OpenCLIP ViT-B/32	Feature extraction
Orchestration	Apache Airflow 3.2.0	Pipeline scheduling and task management
Experiment tracking	MLflow 3.10.0	Metrics, parameters, artifacts, registry
Data versioning	DVC	Data and pipeline reproducibility
Vector search	FAISS `IndexFlatIP`	Efficient similarity retrieval
Search API	FastAPI + uvicorn	REST interface; loose coupling
Model server	MLflow pyfunc serve	Inference serving
Training framework	PyTorch	Model training and evaluation
Metadata database	PostgreSQL 15	Shared state for Airflow and MLflow
Containerisation	Docker Compose v2	Environment parity; service isolation
Monitoring	Prometheus + Grafana	Metrics, dashboards
Alert	Alert Manager	Alerting (MailTrap smtp)

Table 9: Full technology stack with assigned MLOps roles.

14. Quick Start

Prerequisites: Docker and Docker Compose v2; 8 GB RAM minimum (16 GB recommended); approximately 12-15 GB disk space for images, embeddings, and MLflow artifacts.

```
# 1. Place the weekly CSV
mkdir -p data/incoming && cp products.csv data/incoming/week1.
   csv

# 2. Start infrastructure
docker compose up -d

# 3. Trigger the pipeline
#    Open http://localhost:8080 (admin / admin)
#    Enable and trigger: clip_product_retrieval_pipeline
```

To add a new week's data, place the CSV at `data/incoming/week2.csv`. The DAG will detect it on the next 5-minute tick, run the full pipeline, compare against the Week 1 baseline, and retrain only if the drift score exceeds 0.30.